Highly Available Web Application on AWS

Application Load Balancer and Auto Scaling Group (Spot Instances) across two Availability Zones

Project type	Cloud infrastructure (AWS Management Console)
Region	Asia Pacific (Mumbai) - ap-south-1
Services	VPC, EC2, Launch Templates, Application Load Balancer, Auto Scaling, Systems Manager (Session Manager)
Date	October 21, 2026
Prepared by	Parikshit

1. Introduction and Objective

A single web server is a single point of failure: if the instance or its Availability Zone (AZ) goes down, the site goes down. This project builds a web application that stays available by spreading identical servers across two AZs behind a load balancer, and by letting an Auto Scaling group automatically replace servers that fail. Spot Instances are requested to reduce compute cost.

The objectives were to:

- Create a VPC with public subnets in two Availability Zones.
- Build a launch template that installs Apache and serves a page showing the instance ID and AZ that handled the request.
- Place an Application Load Balancer (ALB) in front of the servers, with a target group and HTTP health checks.
- Use an Auto Scaling group to keep the desired number of healthy instances running and to replace unhealthy ones.
- Verify the whole path end to end from a browser and from the command line.

2. Architecture

Users reach the application through the ALB's public DNS name. The ALB lives in both public subnets and forwards HTTP traffic on port 80 to the target group, sending requests only to targets that pass the health check. The Auto Scaling group launches instances from the launch template into the two subnets and registers them with the target group automatically.

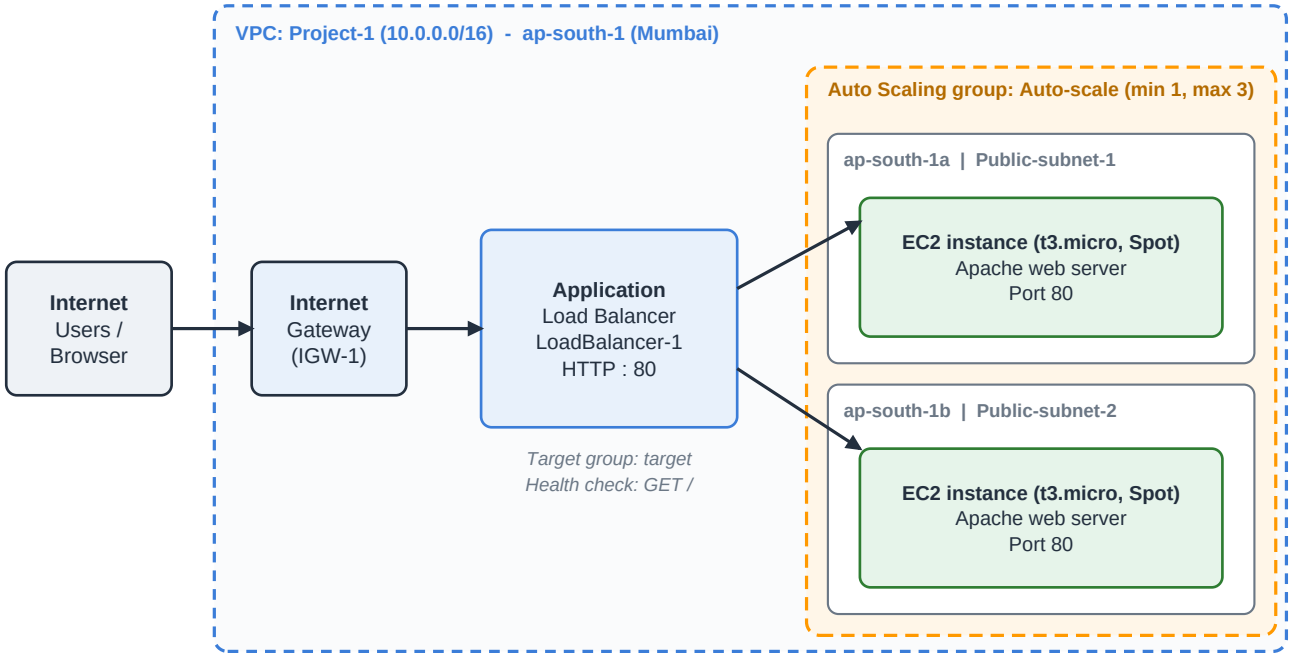


Figure 1. High-level architecture of the project.

3. AWS Resources Used

Resource	Name / ID	Purpose
VPC	Project-1 vpc-0f898a83fb26de100 (10.0.0.0/16)	Isolated virtual network for the project
Subnets	Public-subnet-1 (ap-south-1a) Public-subnet-2 (ap-south-1b)	One public subnet in each of two AZs for fault tolerance
Internet gateway	IGW-1	Gives the VPC a path to and from the internet
Route table	Route-1 (rtb-0db4c739efd734288)	Routes internet traffic to IGW-1; used by both subnets
Launch template	Template-1 (lt-06935821c7687ff73) AMI ami-01a00762f46d584a1, t3.micro, key pair 'test', security group sg-06a07d652ecc3159e	Blueprint for every instance; Spot instances requested
Target group	target (Instance, HTTP : 80)	List of web servers the ALB can send traffic to
Load balancer	LoadBalancer-1 (Application, internet-facing) LoadBalancer-1-1953634195.ap-south-1.elb.amazonaws.com	Single public entry point; spreads traffic across AZs
Auto Scaling group	Auto-scale (limits 1 to 3)	Maintains capacity and replaces unhealthy instances
Systems Manager	Session Manager	Browser-based shell used to inspect instances without SSH

Target group health check settings

Setting	Value	Setting	Value
Protocol	HTTP	Interval	30 seconds
Path	/	Timeout	5 seconds
Port	Traffic port (80)	Healthy threshold	5 consecutive successes
Success codes	200	Unhealthy threshold	2 consecutive failures

4. Implementation

4.1 Network: VPC, subnets and internet access

The VPC **Project-1** (10.0.0.0/16) contains two public subnets, **Public-subnet-1** in ap-south-1a and **Public-subnet-2** in ap-south-1b. Both use the route table **Route-1**, which sends internet traffic to the internet gateway **IGW-1**, as shown in the VPC resource map.

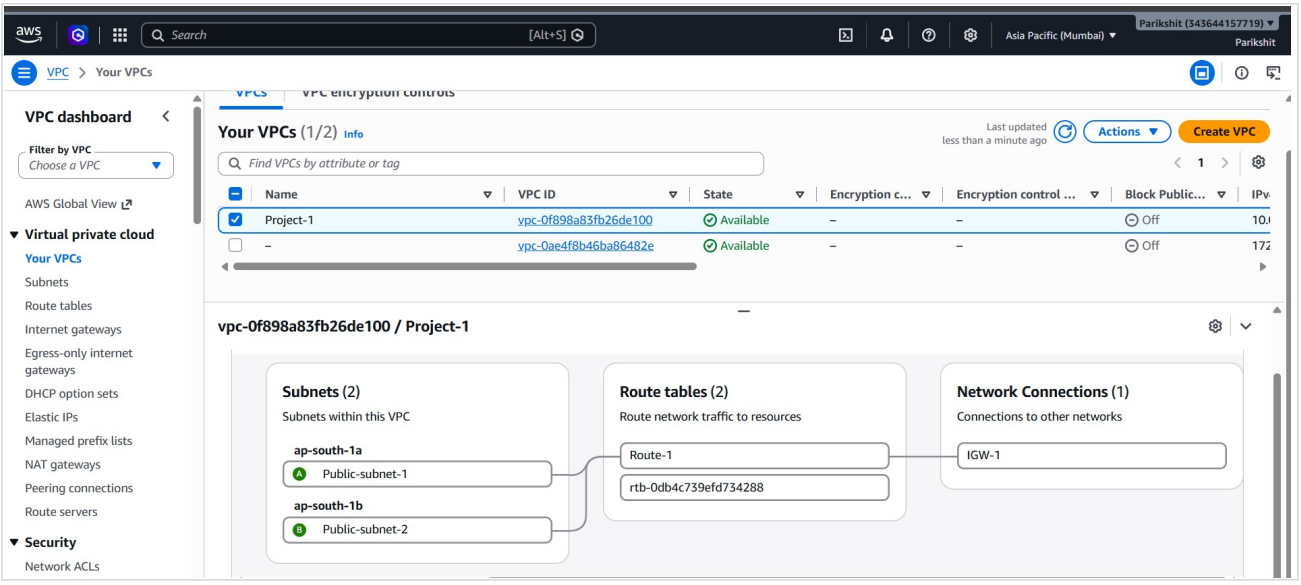


Figure 2. VPC resource map: two subnets in two AZs, one route table and an internet gateway.

4.2 Launch template

Template-1 defines how each web server is created: a t3.micro instance from the selected AMI, the key pair *test*, security group sg-06a07d652ecc3159e, and a request for Spot capacity. Its user data installs Apache and writes a web page containing the instance ID and Availability Zone (see Appendix A). This makes it easy to see which server answered a request.

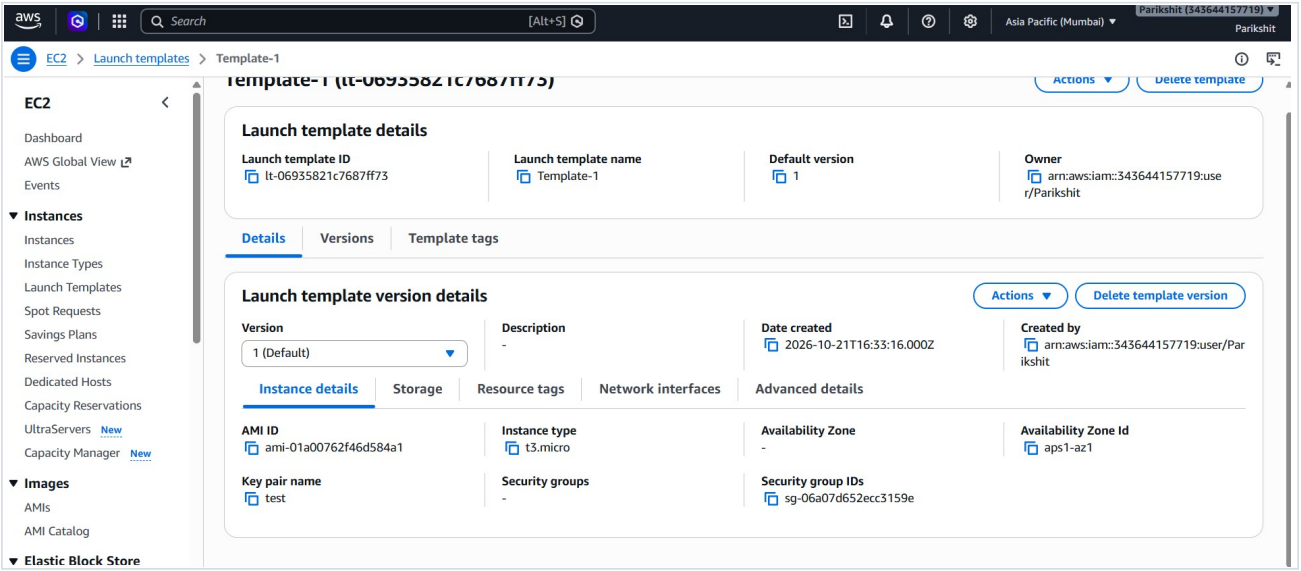


Figure 3. Launch template Template-1 (version 1, default).

4.3 Target group and Application Load Balancer

A target group named **target** (HTTP, port 80) holds the web servers and performs the health checks listed in Section 3. The internet-facing ALB **LoadBalancer-1** is placed in both AZs and its HTTP:80 listener forwards all requests to this target group. Figure 4 shows an earlier load balancer (LB-1) that was still provisioning; the final design uses LoadBalancer-1.

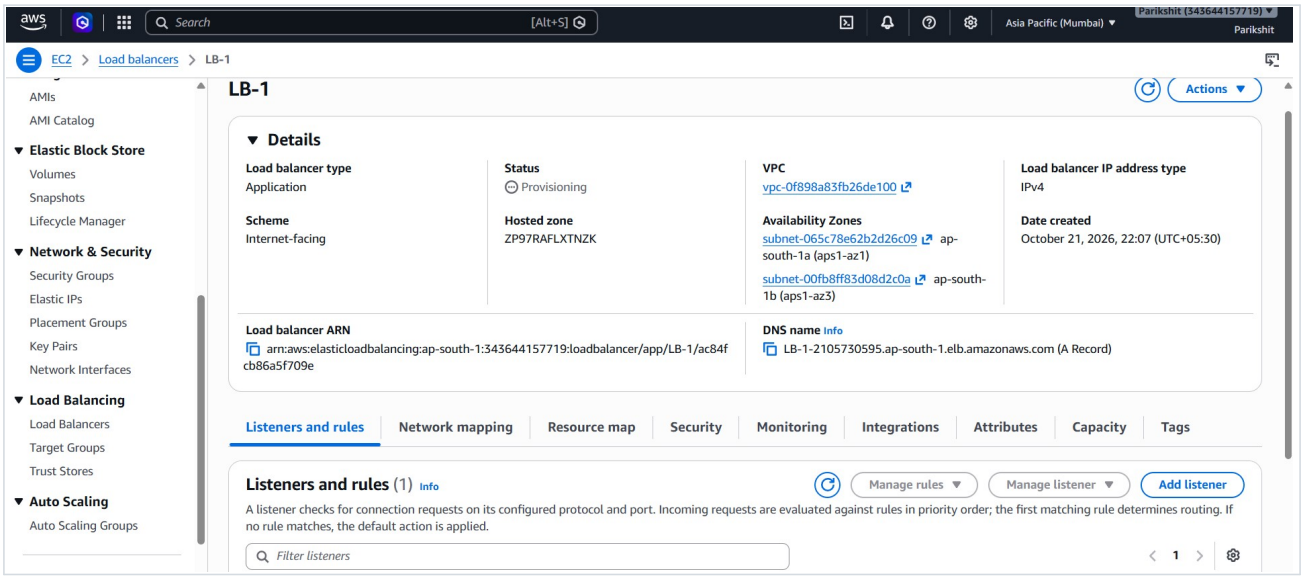


Figure 4. First load balancer attempt (LB-1) in the Provisioning state.

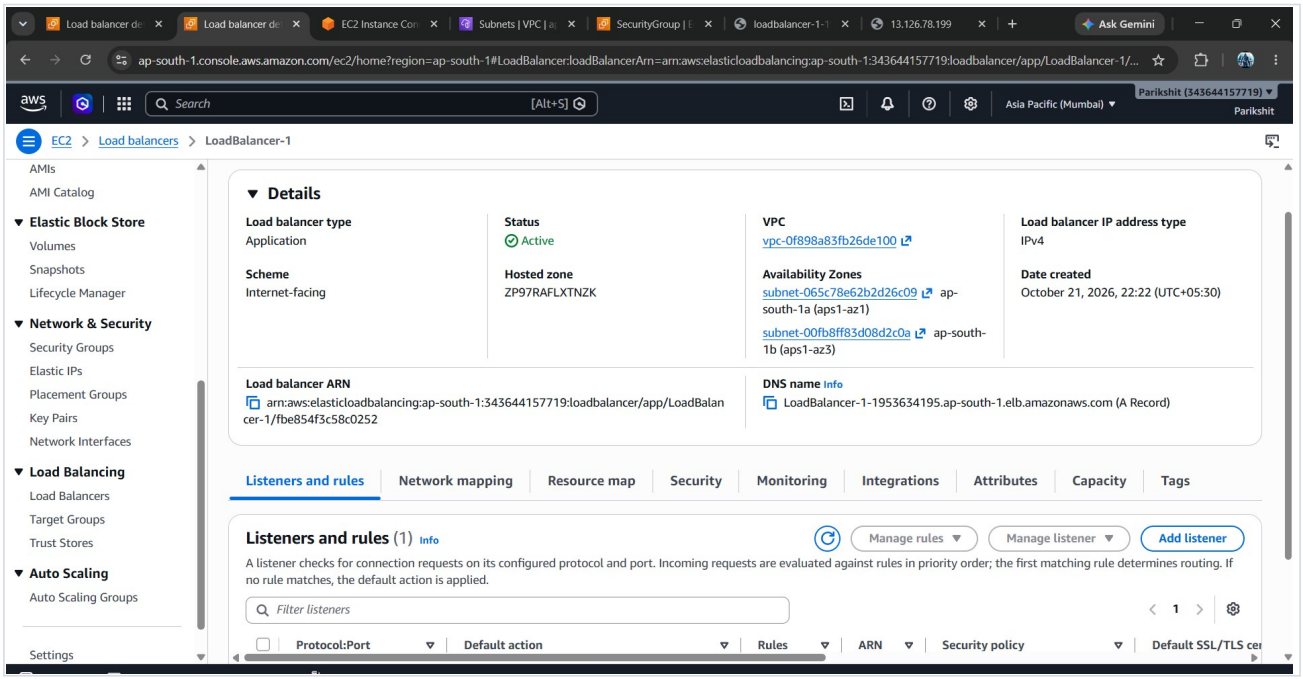


Figure 5. LoadBalancer-1: Application type, internet-facing, status Active, subnets in ap-south-1a and ap-south-1b.

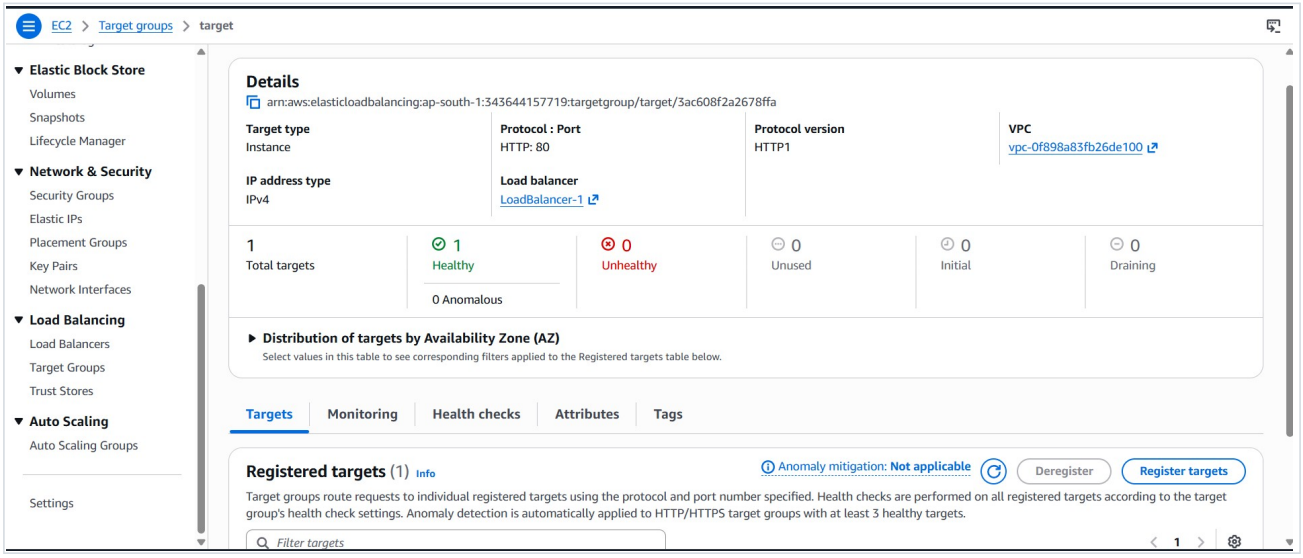


Figure 6. Target group 'target': HTTP port 80, one registered target reporting Healthy.

4.4 Auto Scaling group

The Auto Scaling group **Auto-scale** uses Template-1 and is attached to the load balancer's target group, so new instances are registered automatically and failed ones are replaced. Figure 7 was captured with two instances running (desired capacity 2, both healthy). Figure 8 shows the group's later configuration with scaling limits of 1 to 3 and a desired capacity of 1.

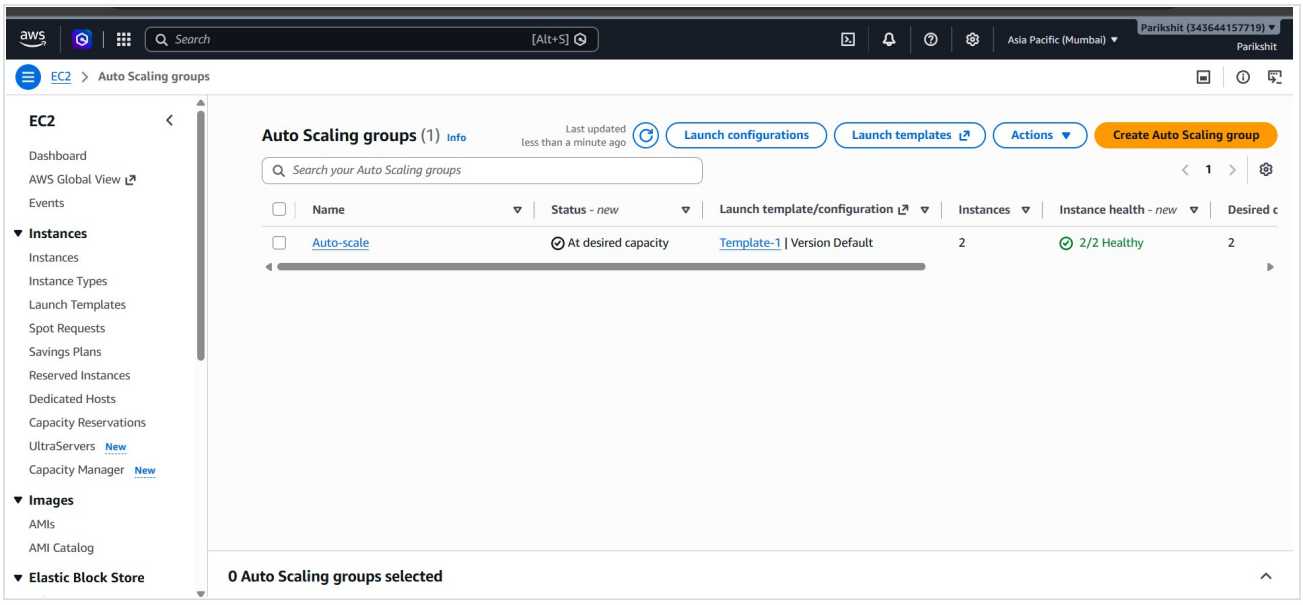


Figure 7. Auto Scaling group at desired capacity: 2 instances, 2/2 healthy.

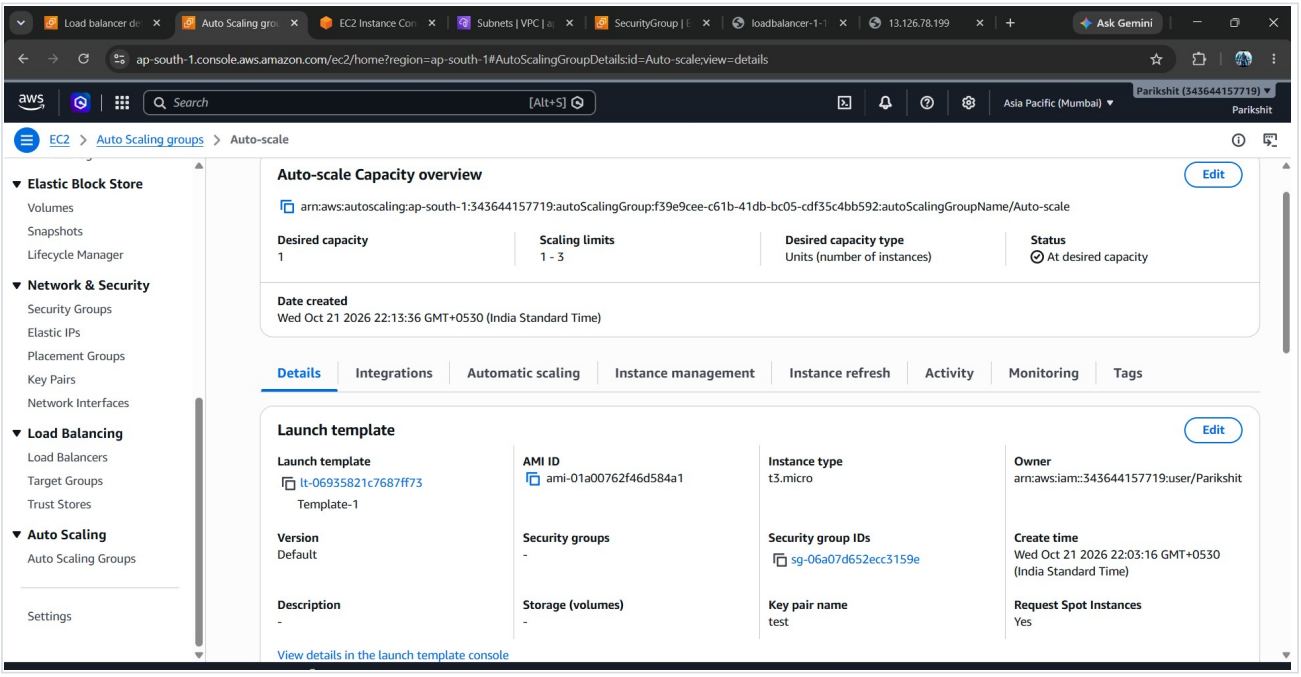


Figure 8. Auto-scale details: launch template Template-1, scaling limits 1 to 3, Spot instances requested.

5. Testing and Verification

The setup was checked in four ways: the load balancer's resource map, the target health status, requests from a browser and from *curl*, and commands run on the instance through Session Manager.

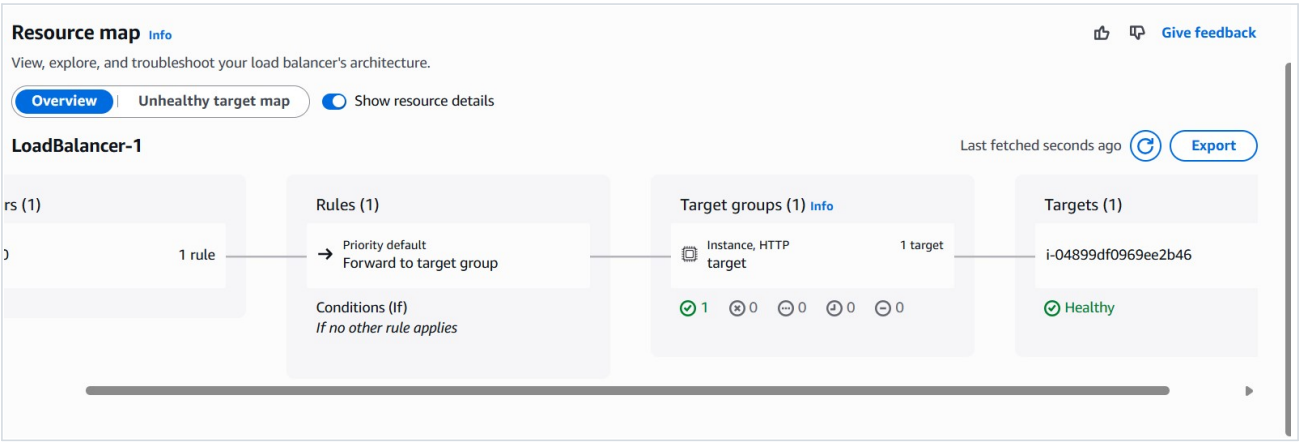


Figure 9. Resource map: listener forwards to the target group, whose target i-04899df0969ee2b46 is Healthy.

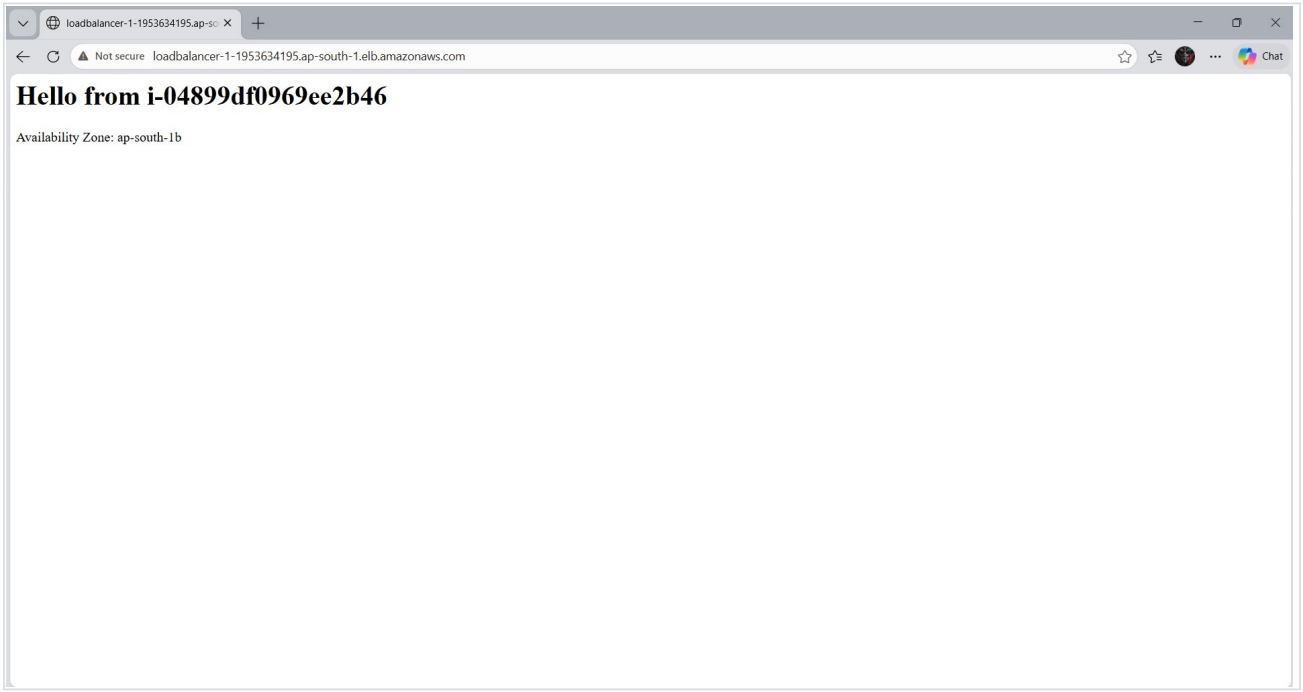


Figure 10. Browser test against the load balancer's DNS name over HTTP: the page served by instance i-04899df0969ee2b46 in ap-south-1b.

Test	Result
Target health in the target group	Registered target shows Healthy (Figures 6 and 9)
curl http://localhost on the instance	HTTP/1.1 200 OK from Apache 2.4.66 (Ubuntu)
curl http://<private IP> on the instance	HTTP/1.1 200 OK, so Apache listens on all interfaces
Firewall and listening port on the instance	ufw inactive; Apache listening on *:80
curl http://<ALB DNS name> from a laptop	HTTP 200 and the page text 'Hello from i-04899df0969ee2b46', 'Availability Zone: ap-south-1b'
Browser request to the ALB over HTTP	Same page displayed (Figure 10)

6. Extended Lab: Private Instances, NAT Gateway and Session Manager

An earlier part of the work (screenshots dated the same day) used two instances, **web-1** in ap-south-1a and **web-2** in ap-south-1b, with no public IPv4 DNS name. A second route table, **Route-2**, sends 0.0.0.0/0 to a NAT gateway (nat-1d48bd815f1d4dcf3) so such instances can reach the internet for package installation without being reachable from it. Access to web-2 was obtained through **Session Manager**, with no SSH port or bastion host, and the session shows the nginx web server installed and running.

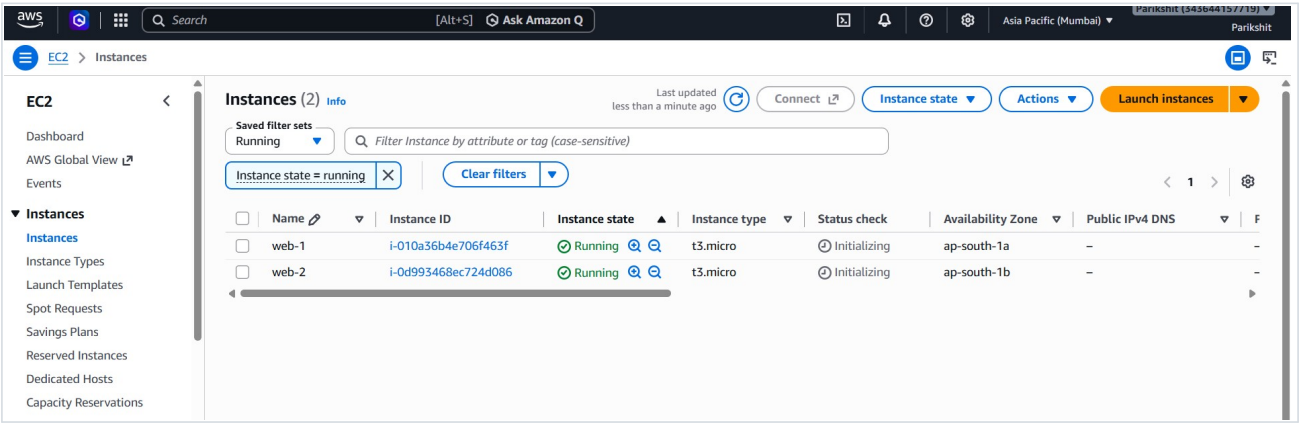


Figure 11. web-1 (ap-south-1a) and web-2 (ap-south-1b) running as t3.micro instances.

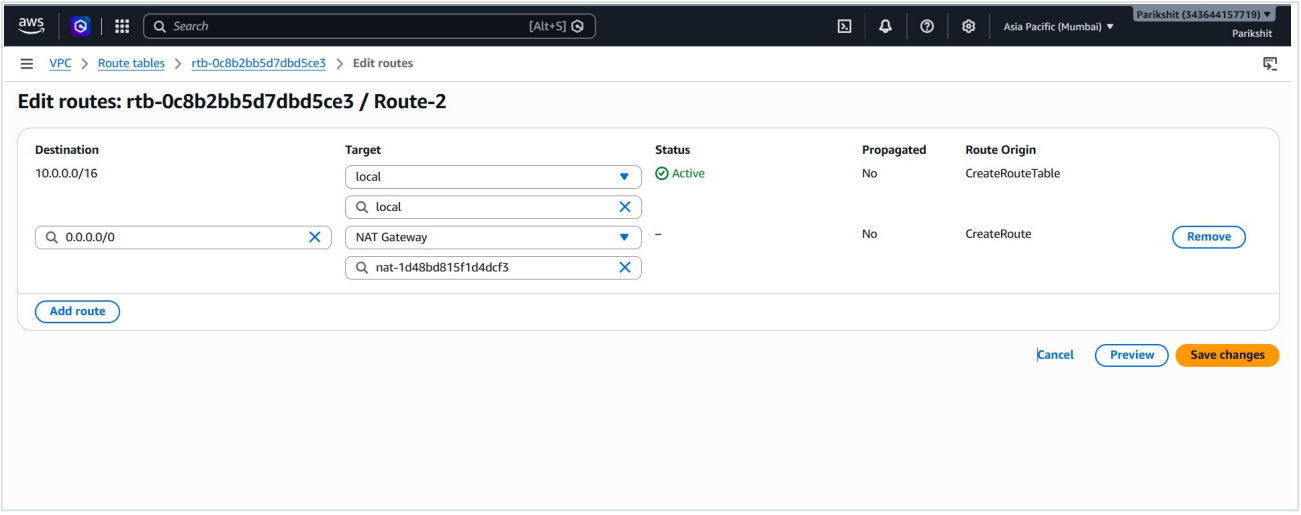


Figure 12. Route-2: local VPC route plus 0.0.0.0/0 pointing to a NAT gateway.

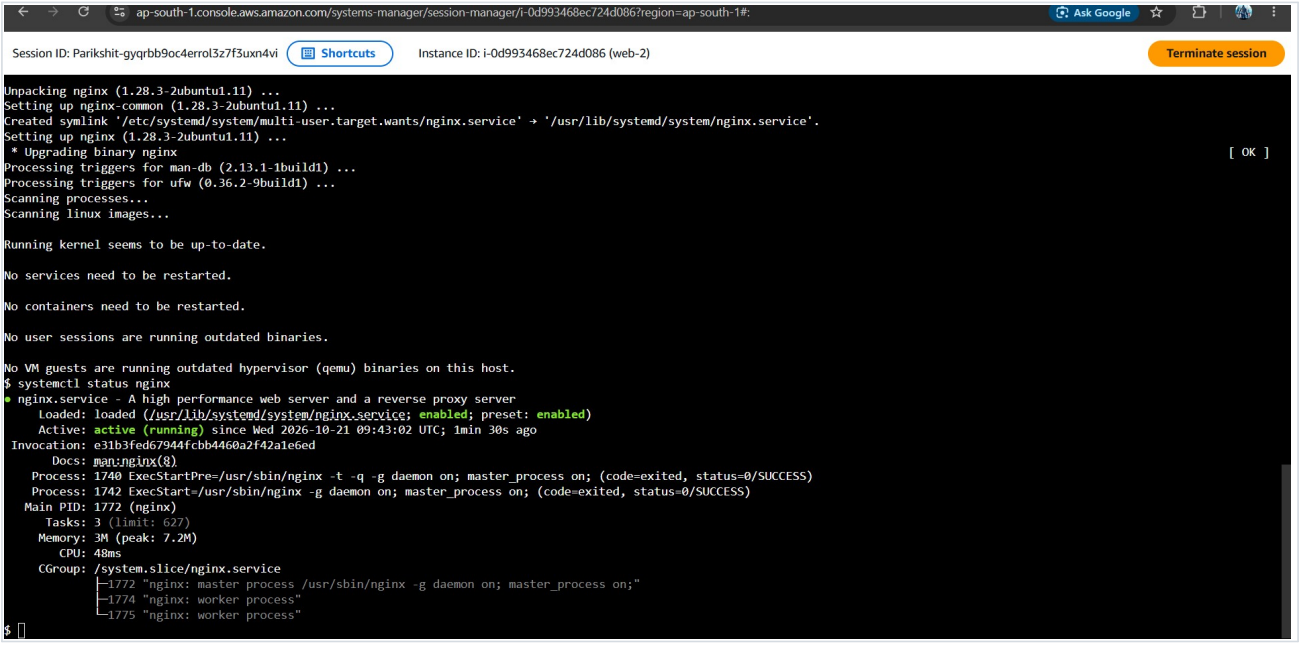


Figure 13. Session Manager shell on web-2 (i-0d993468ec724d086): nginx active and running.

7. Issues Encountered and How They Were Handled

Issue	Investigation	Outcome
Target showed Unhealthy with 'Request timed out'	Checked Apache on the instance (200 OK on localhost and private IP, firewall inactive, port 80 listening), the security groups, the ALB's AZ and subnet mapping, and the health check settings.	The instance was confirmed healthy. The target later reported Healthy (Figure 6). With a threshold of 5 checks at 30-second intervals, status takes about 2.5 minutes to change.
Browser could not connect to the ALB although curl worked	The ALB only has an HTTP:80 listener. Browsers often upgrade a bare address to HTTPS (port 443), which has no listener.	Using the full http:// address loaded the page (Figure 10).
Session Manager showed 'Not connected' on an earlier instance	The SSM agent log reported that no IAM credentials were available for the instance.	Fixed by attaching an IAM role with the AmazonSSMManagedInstanceCore policy; Session Manager then worked (Figure 13).

8. Results and Conclusion

The project delivered a working web application behind an internet-facing Application Load Balancer. Requests sent to the load balancer's DNS name reach an Apache server in the target group, which reports its instance ID and Availability Zone. The target group reports its registered target as Healthy, and the Auto Scaling group keeps the instances registered and replaces unhealthy ones. The extended lab also demonstrated outbound internet access through a NAT gateway and secure shell access with Session Manager.

Key concepts demonstrated: multi-AZ design, load balancing with health checks, self-healing through Auto Scaling, cost reduction with Spot Instances, and access without SSH.

9. Recommendations for Improvement

- **Keep at least 2 instances.** A minimum capacity of 1 cannot survive the loss of that instance; set minimum and desired capacity to 2 so there is one server in each AZ.
- **Tighten security groups.** Allow port 80 on the instance security group only from the load balancer's security group, and remove public SSH.
- **Add HTTPS.** Request a certificate in AWS Certificate Manager and add an HTTPS:443 listener, with a redirect from HTTP.
- **Use private subnets.** Move the instances to private subnets behind the ALB, using a NAT gateway or VPC endpoints for outbound access.
- **Handle Spot interruptions.** Use several instance types and keep one On-Demand base instance so capacity is not lost if Spot is reclaimed.
- **Add monitoring.** Create CloudWatch alarms on unhealthy host count and CPU, with notifications through SNS.
- **Automate.** Define the infrastructure in Terraform so it can be recreated and destroyed consistently.

10. Cleanup

To avoid ongoing charges (the load balancer and NAT gateway bill hourly), delete resources in this order: Auto Scaling group, load balancer, target group, launch template, NAT gateway and its Elastic IP, then the VPC, and finally the IAM role.

Appendix A. Launch Template User Data

Script that installs Apache and writes the page showing the instance ID and Availability Zone (uses the instance metadata service with IMDSv2).

```
#!/bin/bash
apt-get update -y
apt-get install -y apache2
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone)
echo "<h1>Hello from $ID</h1><p>Availability Zone: $AZ</p>" \
  > /var/www/html/index.html
systemctl enable --now apache2
```